

Text as Data 2026 — Complete Study Guide

Past Paper Analysis, 2026 Predictions, and A-Grade Revision Notes

Prepared for Dharun | COMPSCI 5096/5106 | University of Glasgow

May 2026

Contents

How to Use This Guide	4
PART I — PAST PAPER ANALYSIS	5
1. Paper Format Evolution	5
2. The Question-Slot Pattern	5
3. Q1 / Q2 / Q3 Theme Repeats	6
3.1 Question 1 — Sub-Part Breakdown	6
3.2 Question 2 — Sub-Part Breakdown	7
3.3 Question 3 — Sub-Part Breakdown	8
PART II — 2026 PREDICTIONS	12
4. Consolidated 2026 Forecast	12
5. Q1 Prediction — Full Mock Paper	12
6. Q2 Prediction — Likely Structure	13
PART III — A-GRADE REVISION NOTES	15
7. Tokens & Document Similarity	15
7.1 The NLP Pipeline	15
7.2 Encodings	15
7.3 Tokenisation Approaches	15
7.4 Set Similarity Measures	16
7.5 Properties of a Metric	17
7.6 Regular Expressions Reference	18
7.7 N-grams	18
8. Vectors, TF-IDF & Clustering	19
8.1 Term Frequency Variants	19
8.2 Zipf's Law	19
8.3 Worked Example: Build TF-IDF vector (2025 Q1a style)	19
8.4 Vector Similarity (Geometric)	20

8.5 K-Means Clustering (DUE for first appearance!)	20
9. Language Modelling	23
9.1 Probability Foundations	23
9.2 Language Models	23
9.3 Smoothing	23
9.4 LM Evaluation	23
9.5 Text Generation	24
10. Text Classification	26
10.1 Problem Types	26
10.2 Confusion Matrix & Metrics	26
10.3 Multi-Class Metrics	26
10.4 ROC & AUROC	27
10.5 Train/Val/Test & Cross-Validation	27
10.6 Classifier Problems	27
11. Contextual Word Embeddings & Transformers	29
11.1 Three Types of Word Vectors	29
11.2 Self-Attention	29
11.3 BPE (Byte Pair Encoding) — CRITICAL TOPIC	30
11.4 Special Tokens	32
11.5 Stopword Removal — BoW vs Transformer	32
11.6 Transformer Encoder vs Decoder	32
11.7 Pretraining + Fine-tuning	32
12. POS Tagging & Parsing	34
12.1 Hidden Markov Models	34
12.2 Constituency Parsing	34
12.3 Dependency Parsing	35
13. Ethics in NLP	36
13.1 Key Issues	36
13.2 Best Practices	36
14. Information Extraction	37
14.1 Named Entity Recognition (NER)	37
14.2 Entity Linking	37
14.3 Relation Extraction	37
14.4 Coreference Resolution	38
15. Large Language Models	39
15.1 Training Pipeline	39
15.2 Prompt Engineering	39
15.3 LLM Issues	39
16. Latest Research (Lecture 10)	41
16.1 Retrieval-Augmented Generation (RAG)	41
16.2 Lost in the Middle	41
16.3 Tool Use	41
16.4 Knowledge Editing	41

16.5 Reasoning	41
16.6 Agentic AI	42
16.7 Predicted Q3 synthesis question	42
17. Exam Strategy	43
17.1 Time Budget (90 minutes)	43
17.2 Mark-Detail Calibration	43
17.3 Answer Templates	43
17.4 Pitfalls to Avoid	43
17.5 Final-Week Revision Plan	44
Appendix A — Critical Formulas at a Glance	45
Appendix B — Topic to Lecture to Likely Question	45
Appendix C — Highest-ROI Revision Priorities (Ranked)	46

How to Use This Guide

This document is in three parts:

Part I — Past Paper Analysis (Sections 1-3) maps every Q1/Q2/Q3 sub-part across 2022–2025, identifies the recurring theory questions, and explains the structural patterns in the exam.

Part II — 2026 Predictions (Sections 4-6) gives confidence-rated predictions for each question slot, plus a fully-worked predicted Q1 you can attempt under timed conditions.

Part III — Revision Notes (Sections 7-17) covers every lecture with core theory, worked examples, pre-written model answers for the most-repeated exam questions, and exam strategy.

Exam format reminder: Open book, online, 1h 30m. Three questions × 20 marks = 60 marks total. Each mark $\approx$ one distinct idea or point.

PART I — PAST PAPER ANALYSIS

1. Paper Format Evolution

Year	Total	Q1	Q2	Q3	Notes
2022	51	18	15	18	Pre-restructure, uneven marks
2023	60	20	20	20	First “modern” 3×20 format
2024	60	20	20	20	Standardised
2025	60	20	20	20	Standardised
2026 (predicted)	60	20	20	20	Confirmed by Sean MacAvaney in Lecture 10

Takeaway: Treat 2023–2025 as the canonical pattern; 2022 is an outlier with different mark distribution.

2. The Question-Slot Pattern

The course has 10 lectures. Past papers consistently slot them as follows:

Question slot	Typical lecture coverage
Q1	Lecture 1 (Tokens, Similarity) + Lecture 2 (Vectors, TF-IDF, Clustering)
Q2	Lecture 3 (Language Modelling) OR Lecture 4 (Text Classification)
Q3	One advanced/applied topic from Lectures 4–9 — always scenario-based or critique-based, always zero calculation

This pattern has held for the last three years and is the strongest predictor we have.

3. Q1 / Q2 / Q3 Theme Repeats

3.1 Question 1 — Sub-Part Breakdown

Year	Topic	Sub-parts (with marks)
2022 (18m)	Distributional Semantics & Word Embeddings	(a) outlier-detection pseudo-code [5], (b) window size k effects [2], (c) single-pass distributional semantics pseudo-code [5], (d) function-word vector unreliability [2], (e) negative samples in word2vec [2], (f) polysemy in word2vec [2]
2023 (20m)	Tokenisation	(a) word vs token [2], (b) cross-language tokenisers [2], (c) BPE vocab build for “mississippi is pie” [8], (d) BPE vs rule-based trade-offs [4], (e) preprocessing for writing assistant [4]
2024 (20m)	Tokenisation / BPE	(a) stopwords BoW vs transformer [4], (b) [UNK] + Unicode [4], (c) BPE encoding of “mississippi pie” [6], (d) BPE vocab size too low/high [6]
2025 (20m)	TF-IDF & Set Similarity	(a) construct TF-IDF vector [4], (b) reconstruct text from TF-IDF [4], (c) Overlap + Jaccard [4], (d) compute proposed sim measure [2], (e) 3 problems with proposed sim [6]

Q1 Repeating Theory Threads

Theme	2022	2023	2024	2025	Repeat strength
BPE / sub-word tokenisation theory	—	yes (c,d)	yes (b,c,d)	—	Very high

Theme	2022	2023	2024	2025	Repeat strength
Preprocessing—choice in context (stem/lemma/stopword)	—	yes (e)	yes (a)	—	High
Cross-language tokenisation / Unicode	—	yes (b)	yes (b)	—	High
Set similarity calculations (Jaccard, Dice, Overlap)	yes (Q2d)	—	—	yes (c)	Medium
Properties of a metric / semi-metric	yes (Q2d)	—	—	yes (e)	Medium
TF-IDF construction	—	—	—	yes (a)	Low (only 2025)

3.2 Question 2 — Sub-Part Breakdown

Year	Topic	Sub-parts (with marks)
2022 (15m)	Word frequencies & Language Models	(a) trigrams >1 in alien text [3], (b) max theoretical n-grams $n=1..5$ [3], (c) explain tokenisation rules [1+1], (d-i) Dice + Jaccard between probes [4], (d-ii) Dice as semi-metric / triangle inequality [3]
2023 (20m)	Language Models (bigram)	(a) perplexity under Model X/Y [6], (b) greedy + beam search $b=2$ from prefix [8], (c) why smoothing, which values benefit, negatives [3], (d) problems of small/large n , how to choose [3]

Year	Topic	Sub-parts (with marks)
2024 (20m)	Language Modelling with Character N-grams	(a) unigram + greedy next 3 [2], (b) trigram + greedy next 3 [6], (c) bigram + beam search b=2 next 2 [8], (d) propose text bigram can't generate + fix [4]
2025 (20m)	Text Classification (mode switch!)	(a) 4 problems with recipe-search labelling [8], (b) 4 effects of triplicated samples [4], (c) identify 8 anomalies in metrics tables [8]

Q2 has TWO modes

This is the most important pattern: **Q2 is either Language Modelling OR Text Classification.**

- **LM mode (2022, 2023, 2024):** N-grams, perplexity, greedy + beam search, smoothing
- **Classification mode (2025):** Metrics, data issues, anomaly-spotting

2025 was the first time it flipped. **No clean rule about which comes next.**

Q2 Repeating Theory Threads

Theme	2022	2023	2024	2025	Repeat strength
Bigram/trigram model construction	—	yes	yes (b,c)	—	High (when LM mode)
Perplexity calculation	—	yes (a)	—	—	Medium
Greedy generation	—	yes (b)	yes (a,b)	—	High (when LM mode)
Beam search (b=2)	—	yes (b)	yes (c)	—	Very high (when LM mode)
Smoothing (add-k, why, downsides)	—	yes (c)	yes (d)	—	High (when LM mode)
Choosing n in n-gram models	—	yes (d)	—	—	Medium
Classifier evaluation metrics critique	—	—	—	yes (c)	New theme
Class imbalance / data leakage	—	—	—	yes (b)	New theme
Multi-class vs multi-label	—	—	—	yes (a)	New theme

3.3 Question 3 — Sub-Part Breakdown

Year	Topic	Sub-parts (with marks)
2022 (18m)	NLP / Patent Classification	(a) justify case-norm + stemmer with example [4], (b) POS-tag-aware feature example [3], (c) why HMM POS-tagger fails on domain text + fix [4], (d) interpret train/test precision/recall [3], (e) inject POS into BERT without ensemble [4]
2023 (20m)	Factuality of LM Responses (critique GPT-3)	(a) critique “two probabilities LMs measure” [3], (b) critique TF vs IDF [2], (c) critique dimensionality reduction [4], (d) critique HMM for POS tagging [3], (e) critique nested entity mentions [3], (f) how GPT-3 encodes knowledge + 2 challenges [5]
2024 (20m)	Indirect Answer Classification (Louis et al. EMNLP 2020)	(a) zero-shot vs data contamination [2], (b) 3 counter-arguments to LLM labels (Fleiss $\kappa=0.61$) [6], (c) text-similarity eval insufficient for unseen scenarios [6], (d) motivation for multi-stage fine-tuning [6]
2025 (20m)	Sentiment Analysis (BeStep scenario)	(a) rank LogReg/BERT/LLaMA by cost [2], (b) procedure to determine most effective [4], (c) cross-lingual LogReg + BERT limits + solution [4], (d) sub-word false positive (brand “BeStep”) [4], (e) extending to per-product sentiment [6]

Q3 Repeating Theory Threads

Theme	2022	2023	2024	2025	Repeat strength
HMM for POS tagging	yes (c)	yes (d)	—	—	High (overdue!)
BERT classifier	yes (e)	—	yes (a)	yes (a,c,d)	Very high
limitations / behaviour	—	—	—	yes (d)	Medium
Sub-word tokenisation	—	—	—	yes (d)	Medium
real-world issue	—	—	—	—	Medium
LLM factuality / hallucination / knowledge	—	yes (a-f)	—	—	Medium
Test set leakage / data contamination	—	—	yes (a)	—	Medium
Cross-lingual model issues	—	—	—	yes (c)	Medium
Cost/complexity-ranking (LogReg < BERT < LLM)	—	—	—	yes (a)	New
Annotation quality / Fleiss kappa	—	—	yes (b)	—	Low
Fine-tuning vs zero-shot / multi-stage	—	—	yes (a,d)	—	Medium
Information Extraction (NER, EL, RE)	—	—	—	—	0 — never asked!
Ethics, copyright, environmental cost	—	—	—	—	0 — never asked!
K-Means clustering	—	—	—	—	0 — never asked!

Theme	2022	2023	2024	2025	Repeat strength
RAG / retrieval-augmented LMs	—	—	—	—	0 — never asked!

Q3 Style Signature

Q3 is **always scenario-based or critique-based**: - 2022: “You work at the Intellectual Property Office...” (scenario) - 2023: “Critique these GPT-3 responses...” (critique) - 2024: “Recall the paper by Louis et al...” (paper-based) - 2025: “Working for BeStep, a brand...” (scenario)

PART II — 2026 PREDICTIONS

4. Consolidated 2026 Forecast

Question	Predicted topic	Confidence	Key sub-parts likely
Q1	Tokenisation + Set Similarity hybrid	~70%	preprocessing-in-context, Unicode/BPE connection, BPE rule application, Jaccard/Dice calculation, metric properties
Q2	Language Modelling (bigram/trigram)	~55%	n-gram model build, beam search b=2 , smoothing theory, perplexity OR impossible-sequence question
Q3	LLM scenario (RAG, factuality, hallucination)	~40%	RAG vs fine-tune trade-offs, hallucination causes + mitigations, prompt engineering, data contamination, cost analysis

Wild cards to revise as backups:

- **Q1 backup:** K-Means clustering (on crib sheet, never tested)
- **Q2 backup:** Text Classification (continuing the 2025 mode-switch)
- **Q3 backup #1:** Information Extraction (own lecture, never tested)
- **Q3 backup #2:** POS Tagging + HMM Viterbi (overdue since 2023)
- **Q3 backup #3:** Ethics (own lecture, never tested as main topic)

5. Q1 Prediction — Full Mock Paper

Predicted Question 1 — 2026 (20 marks)

Topic: Tokenisation and Document Similarity

You are building a search system for cooking recipes written by users on a community website. Recipes are short, often contain abbreviations (e.g. “tbsp”, “1/2 c”), emojis, and mix English with loan-words from other languages (e.g. “ragù”, “jalapeño”, “yakisoba”).

(a) Within this recipe-search context, justify whether you would use **stemming**, **lemmatization**, and **stopword removal** in your preprocessing pipeline. Provide a concrete recipe-domain example for each of the three decisions. **[4]**

(b) A teammate suggests using a simple whitespace tokeniser. Considering what you know about **Unicode encoding** and the multilingual nature of the recipe corpus, explain two specific problems this will cause and propose one tokenisation method that addresses both. **[4]**

(c) Perform sub-word tokenisation over the following two tokens using the BPE rules below. Assume all characters [t, o, m, a, e, s, u, c, p] exist in the vocabulary. **Show all your steps.** **[6]**

BPE Rules (applied in order):

```
t, o    -> to
m, a    -> ma
to, ma  -> toma
a, t    -> at
o, e    -> oe
toma, t -> tomat
tomat, oe -> tomatoe
s, a    -> sa
sa, u    -> sau
sau, c   -> sauc
sauc, e  -> sauce
```

Tokens to encode: tomatoes sauce

(d) Compute the **Sørensen-Dice Coefficient**, **Jaccard Similarity**, and **Overlap Coefficient** between the following two tokenised recipe ingredient lists. Show your work. **[4]**

Recipe A: tomato, onion, garlic, basil, salt, olive_oil

Recipe B: tomato, garlic, oregano, salt, olive_oil, pepper, chilli

(e) A colleague proposes that “if Sørensen-Dice gives a similarity, then $1 - \text{dice}$ must be a valid distance metric”. Using your answer from (d) and one additional small example, demonstrate whether this claim holds. State which property of a metric (if any) is violated. **[2]**

6. Q2 Prediction — Likely Structure

Scenario A (~55% confidence) — Language Modelling returns

Likely sub-parts: - (a) Build n-gram counts from a small text and compute one probability [4-6] - (b) Apply **beam search** with beam width 2 for next 2-3 tokens — VERY high repeat [6-8] - (c) Why is **smoothing** applied? Which values benefit most? Negatives of too much smoothing? [3] - (d) Compute **perplexity** of a given sequence under the model [4-6] **or** explain why a specific sequence is impossible to generate and how to fix it [3-4]

Scenario B (~40% confidence) — Text Classification (continuing 2025)

Likely sub-parts: - (a) 4 problems with a flawed labelling pipeline scenario [6-8] - (b) Identify N anomalies in given confusion matrices/metrics tables [6-8] - (c) Choose appropriate metric (precision vs recall vs F1) for a scenario; justify [4] - (d) ROC curve interpretation OR k-fold cross-validation justification [4]

Tilt toward Scenario A because: LM appeared 3 years in a row before 2025, beam search

+ smoothing are the most marks-dense theory questions, and the crib sheet devotes more space to LM than classification.

PART III — A-GRADE REVISION NOTES

How to use: Each topic has (1) core theory, (2) at least one worked example, (3) common pitfalls, and (4) model answers for repeated exam questions. The “Exam-style Q&A” boxes are A-grade templates — practise reproducing them in your own words.

7. Tokens & Document Similarity

7.1 The NLP Pipeline

Standard preprocessing pipeline:

1. **Data Cleaning** — strip HTML, links, boilerplate, tables
2. **Tokenisation** — split text into tokens (and often sentences)
3. **Normalisation** — reduce variation:
 - **Case folding** (“Textile” → “textile”)
 - **Stemming** — rule-based suffix stripping (“running” → “run”). Fast, crude.
 - **Lemmatisation** — dictionary lookup to base form (“better” → “good”). Slower, accurate.
4. **Stopword removal** — drop high-frequency low-info words (“the”, “is”, “of”)

Key principle: every step is a *choice*. Justify it for the application.

A-grade reasoning template

“For [task], I would/wouldn’t use [step] because [task-specific consequence]. For example, [concrete example from the domain].”

7.2 Encodings

Encoding	Bytes per char	Coverage
ASCII	1 byte	128 chars (English + symbols)
Unicode (UTF-8)	1–4 bytes (variable)	150,000+ chars (all world scripts, emojis, maths)

Key fact: Unicode is **variable-length** (1–4 bytes per character). UTF-8 encodes ASCII chars in 1 byte but Chinese chars in 3, emojis in 4.

7.3 Tokenisation Approaches

Method	How it works	Strength	Weakness
Whitespace	split on spaces	trivial	fails on punctuation, no-space languages, contractions
Rule-based (regex)	hand-crafted patterns	precise, interpretable	brittle, language-specific
Subword (BPE, WordPiece)	learned from corpus	handles OOV, multilingual	needs training data

Exam-style Q&A: word vs token (2023 Q1a)

Q: What is the difference between a word and a token? Why do text processing systems usually operate over tokens instead of words?

A: A *word* is a linguistic unit defined by meaning and orthography (e.g. “don’t”, “São Paulo”). A *token* is an algorithmically-produced unit — what comes out of the tokeniser — which may be a word, subword, punctuation mark, or symbol. Systems operate over tokens because (1) tokens are unambiguous and reproducible by code, (2) subword tokenisers handle unseen words gracefully, and (3) tokens align with the units the downstream model was trained on.

Exam-style Q&A: why tokenisers differ across languages (2023 Q1b)

A: Languages have different orthographic conventions. (1) Chinese, Japanese, and Thai have no spaces between words, so whitespace tokenisation fails entirely. (2) German compounds words (“Donaudampfschiffahrtsgesellschaft”) — an English tokeniser would treat this as one token, losing semantic structure. (3) Arabic and Hebrew use clitics attached to roots that should be split off. An English tokeniser applied to Chinese would treat an entire sentence as a single token.

7.4 Set Similarity Measures

Given documents as sets of tokens X and Y :

Measure	Formula	Range
Overlap count	$\ X \cap Y\ $	$[0, \min(\ X\ , \ Y\)]$
Overlap Coefficient	$\frac{\ X \cap Y\ }{\min(\ X\ , \ Y\)}$	$[0, 1]$
Sørensen-Dice	$\frac{2 \cdot \ X \cap Y\ }{\ X\ + \ Y\ }$	$[0, 1]$
Jaccard	$\frac{\ X \cap Y\ }{\ X \cup Y\ }$	$[0, 1]$

Relationship: $\text{similarity}(X, Y) \propto -\text{distance}(X, Y)$. Convert via $d = 1 - \text{sim}$.

Worked Example: Set similarity calculation

Recipe A = {tomato, onion, garlic, basil, salt, olive_oil} $\rightarrow |A| = 6$ **Recipe B** = {tomato, garlic, oregano, salt, olive_oil, pepper, chilli} $\rightarrow |B| = 7$

- Intersection: {tomato, garlic, salt, olive_oil} $\rightarrow |A \cap B| = 4$
- Union: {tomato, onion, garlic, basil, salt, olive_oil, oregano, pepper, chilli} $\rightarrow |A \cup B| = 9$

Calculations:

- **Overlap** = $4 / \min(6, 7) = 4/6 = 0.667$
- **Dice** = $2 \cdot 4 / (6 + 7) = 8/13 = 0.615$
- **Jaccard** = $4/9 = 0.444$

Sanity check: Overlap $\geq$ Dice $\geq$ Jaccard always (denominators grow).

7.5 Properties of a Metric

A function d is a **metric** if it satisfies ALL four:

1. **Identity:** $d(X, X) = 0$
 2. **Positivity:** $d(X, Y) > 0$ when $X \neq Y$
 3. **Symmetry:** $d(X, Y) = d(Y, X)$
 4. **Triangle inequality:** $d(X, Z) \leq d(X, Y) + d(Y, Z)$
- **Semi-metric** = violates triangle inequality (e.g. $1 - \text{Dice}$)
 - **Pseudo-metric** = violates positivity ($d(X, Y) = 0$ possible even when $X \neq Y$)

Worked Example: Proving Dice is a semi-metric (2022 Q2d style)

$A = \{a, eda, bcda, bcda, bce\}$, $B = \{ca, eda, bcba, bcda, bce\}$, $C = \{beda, bcda, bceca, ebda, b\}$

- $|A| = 5, |B| = 5, |C| = 5$
- $A \cap B = \{eda, bcda, bce\} \Rightarrow \text{Dice}(A, B) = 6/10 = 0.6$, distance = 0.4
- $A \cap C = \{bcda\} \Rightarrow \text{Dice}(A, C) = 2/10 = 0.2$, distance = 0.8
- $B \cap C = \{bcda\} \Rightarrow \text{Dice}(B, C) = 2/10 = 0.2$, distance = 0.8

Key insight for exam: For a semi-metric proof, you need to construct three sets where $d(A, C) > d(A, B) + d(B, C)$. With Dice, this is possible for sufficiently dissimilar configurations.

Exam-style Q&A: 3 problems with a similarity measure (2025 Q1e)

Generic answer template — adapt to the specific measure given:

1. **Range / direction confusion** — does it return higher for more similar or more distant? Make this explicit.
2. **Symmetry** — does $\text{sim}(A, B) = \text{sim}(B, A)$? Asymmetric measures cause downstream issues.
3. **Division by zero** — what happens when one set is empty, or $A \cap B = \emptyset$?

4. **Triangle inequality** — does it form a true metric? If not, you can't use efficient indexing structures.
5. **Unbounded values** — is there an upper bound?
6. **Sensitivity to set size** — does it score short documents unfairly?

7.6 Regular Expressions Reference

Pattern	Meaning
.	any character
[abc]	one of a, b, c
[^abc]	any char except a, b, c
[a-z]	range
\d \w \s	digit, word char, whitespace
\D \W \S	negated versions
* + ?	0+, 1+, 0 or 1
^ \$	start/end of string
\b	word boundary

7.7 N-grams

An **n-gram** is a contiguous sequence of n items from text.

Theoretical max n-grams for alphabet size A and text length L :

- If $A^n \leq L - n + 1$: max = A^n (alphabet-limited)
- If $A^n > L - n + 1$: max = $L - n + 1$ (text-limited)

Worked Example: Theoretical max n-grams (2022 Q2b)

Alphabet = 5 chars, text length $L = 593$:

n	A^n	$L - n + 1$	max
1	5	593	5 (alphabet-limited)
2	25	592	25
3	125	591	125
4	625	590	590 (text-limited)
5	3125	589	589

Key insight: as n grows, you flip from alphabet-limited to text-limited.

8. Vectors, TF-IDF & Clustering

8.1 Term Frequency Variants

Method	Formula
Raw TF	$\text{tf}(t,d) = \text{count of } t \text{ in } d$
Log-transformed TF	$1 + \log_{10}(tf)$ if $tf > 0$, else 0
IDF	$\log(N/df(t))$
TF-IDF	$(1 + \log(tf)) \cdot \log(N/df)$

Why log on TF? Diminishing returns — the 100th occurrence of “the” shouldn’t count 100× more than the 1st.

Why IDF? Down-weight common words (stopwords get $IDF \approx 0$), boost rare-but-meaningful words.

8.2 Zipf’s Law

In any natural language corpus: $\text{frequency}(\text{rank}) \propto 1/\text{rank}$. The most common word is roughly twice as frequent as the 2nd, 3× as frequent as the 3rd. A tiny set of stopwords dominate any text.

8.3 Worked Example: Build TF-IDF vector (2025 Q1a style)

Vocabulary with IDF:

Token	A	B	C	D	E	F	G	H	I	J
IDF	2	4	1	3	6	4	1	3	3	5

Document: G C G A H A G

Step 1 — Count tokens: A: 2, C: 1, G: 3, H: 1

Step 2 — Apply TF × IDF (linear TF):

- A: $2 \times 2 = 4$
- C: $1 \times 1 = 1$
- G: $3 \times 1 = 3$
- H: $1 \times 3 = 3$
- Others: 0

Vector: [4, 0, 1, 0, 0, 0, 3, 3, 0, 0]

Reverse: reconstruct doc from TF-IDF (2025 Q1b style)

Given [6, 8, 0, 6, 6, 0, 2, 3, 3, 10]:

Token	TF-IDF	IDF	Count
A	6	2	3
B	8	4	2
D	6	3	2
E	6	6	1
G	2	1	2
H	3	3	1
I	3	3	1
J	10	5	2

Tokens (multiset): A A A B B D D E G G H I J J **Ambiguity:** Word order is lost — could be any permutation.

8.4 Vector Similarity (Geometric)

Euclidean distance: $d(\vec{x}, \vec{y}) = \sqrt{\sum_i (x_i - y_i)^2}$

Cosine similarity: $\cos(\theta) = \frac{\vec{x} \cdot \vec{y}}{\|\vec{x}\| \cdot \|\vec{y}\|}$

	Euclidean	Cosine
Returns	distance (0 = same)	similarity (1 = same direction)
Sensitive to magnitude?	Yes	No (length-normalised)
Best for	when magnitudes matter	text (doc length shouldn't matter)

Worked Example: Cosine similarity

$$\vec{x} = [3, 4], \vec{y} = [4, 3]$$

- Dot: $3 \cdot 4 + 4 \cdot 3 = 24$
- $\|\vec{x}\| = \sqrt{25} = 5$
- $\|\vec{y}\| = \sqrt{25} = 5$
- $\cos(\theta) = 24/25 = 0.96$

8.5 K-Means Clustering (DUE for first appearance!)

Goal: partition data into k clusters by minimising within-cluster distance.

Algorithm:

- **Step 0:** Initialise k centroids
- **Step 1:** Assign each point to its nearest centroid
- **Step 2:** Recompute each centroid as the mean of its assigned points
- Repeat 1-2 until convergence

Initialisation methods

Method	How
Random selection / Forgy	Pick k random points as centroids
Random Partition	Randomly assign each point to a cluster, compute centroid
k-means++	Pick centroids far apart (better convergence)

Why init matters: K-means is sensitive to initialisation — can converge to local minima. Run multiple times, pick the best.

Choosing k: the Elbow Method

Plot within-cluster sum of squares (WCSS) for $k = 1, 2, 3, \dots$. Look for the “elbow” where additional clusters give diminishing returns.

Measuring Clustering Quality

Metric	Formula	Interpretation
Cohesion(i)	avg distance of i to same-cluster points	low = tight
Separation(i)	min(avg distance of i to another-cluster points)	high = well-separated
Silhouette(i)	piecewise (see below)	range $[-1, 1]$, higher = better

Silhouette:

- If cohesion < separation: $s = 1 - cohesion/separation$
- If cohesion = separation: $s = 0$
- If cohesion > separation: $s = separation/cohesion - 1$

Interpretation: +1 well-clustered, 0 border, -1 wrong cluster.

Worked Example: K-Means iteration

Data: (1,1), (1,2), (5,4), (6,5). **Init:** C1 = (1,1), C2 = (5,4).

Iteration 1:

- (1,1): C1 (dist 0) vs C2 (dist 5) → C1
- (1,2): C1 (dist 1) vs C2 (dist 4.47) → C1
- (5,4): C1 (dist 5) vs C2 (dist 0) → C2
- (6,5): C1 (dist 6.4) vs C2 (dist 1.41) → C2

New centroids: C1 = (1, 1.5), C2 = (5.5, 4.5)

Iteration 2: assignments unchanged → converged.

Exam-style Q&A: When does K-Means struggle?

1. **Non-spherical clusters** — assumes Euclidean → round clusters
2. **Different cluster sizes/densities** — tends to equal-sized clusters
3. **Outliers** — pull centroids; use K-medoids
4. **Wrong k** — use elbow or silhouette
5. **High-dimensional data** — curse of dimensionality

9. Language Modelling

9.1 Probability Foundations

- $0 \leq P(x) \leq 1, \sum P(x) = 1$
- **Bayes:** $P(A|B) = P(B|A) \cdot P(A)/P(B)$
- **Chain rule:** $P(A, B, C) = P(A|B, C) \cdot P(B|C) \cdot P(C)$
- **Independence:** $P(A|B) = P(A)$ and $P(B|A) = P(B)$

9.2 Language Models

Goal: estimate $P(w_1, \dots, w_n)$ or $P(w_{k+1}|w_1 \dots w_k)$.

Markov assumption (only previous $n-1$ tokens matter):

n	Name	Conditions on
1	unigram	nothing
2	bigram	previous 1 token
3	trigram	previous 2 tokens

Trade-off: Small n = high bias, low variance. Large n = low bias, high variance.

9.3 Smoothing

Why? If a bigram is unseen, MLE gives $P = 0$, zeroing the whole sequence.

Add-k:
$$P_{add-k}(t|\theta) = \frac{\text{count}(t, \theta) + k}{\sum_{t'} \text{count}(t', \theta) + k|V|}$$

Interpolation:
$$P_{interp}(t_i|t_{i-1}, t_{i-2}) = \lambda_1 P(t_i|t_{i-1}, t_{i-2}) + \lambda_2 P(t_i|t_{i-1}) + \lambda_3 P(t_i)$$

Jelinek-Mercer:
$$P_{JM}(t|\theta_d, \theta_c) = \lambda P(t|\theta_d) + (1 - \lambda) P(t|\theta_c)$$

Exam-style Q&A: Why smoothing? (2023 Q2c)

A: Without smoothing, any unseen n -gram in test data gets probability 0, which zeros out the entire sequence probability (and gives infinite perplexity). Smoothing redistributes a small amount of probability mass to unseen events.

Which values benefit most? Cells with zero or very low counts — the rare or unseen transitions.

Negatives of too much smoothing: flattens the distribution, washing out the actual signal — frequent patterns get under-weighted, rare events over-weighted. The model loses discriminative power.

9.4 LM Evaluation

- **Intrinsic:** perplexity, cross-entropy, entropy
- **Extrinsic:** plug into downstream task

Formulas:

- Cross-entropy: $H = -\frac{1}{n} \sum \log_2 P(w_i)$
- Perplexity: $PP = 2^H = (\prod P(w_i))^{-1/n}$
- Entropy: $-\sum P(x) \log_2 P(x)$

Worked Example: Perplexity calculation (2023 Q2a)

Sequence: <S> a b b a <E> (5 transitions)

Model X: $P(a)=0.1, P(b|a)=0.3, P(b|b)=0.4, P(a|b)=0.3, P(|a)=0.5$

$$P = 0.1 \times 0.3 \times 0.4 \times 0.3 \times 0.5 = 0.0018$$

$$PP_X = (0.0018)^{-1/5} = (1/0.0018)^{0.2} = 555.56^{0.2} \approx 3.51$$

Model Y: $P(a)=0.3, P(b|a)=0.1, P(b|b)=0.6, P(a|b)=0.3, P(|a)=0.1$

$$P = 0.3 \times 0.1 \times 0.6 \times 0.3 \times 0.1 = 0.00054$$

$$PP_Y = (0.00054)^{-1/5} \approx 4.55$$

Conclusion: Model X better represents the sequence (lower perplexity = less surprised).

9.5 Text Generation

Method	How
Greedy	Pick highest-probability token at each step
Sampling	Sample from distribution
Beam search	Keep top-k partial sequences
Top-k	Sample only from top k tokens
Top-p (nucleus)	Sample from smallest set with cumulative probability $\geq p$

Worked Example: Beam search b=2 (2023 Q2b style)

Setup: prefix <S> a c, beam width 2. Extend with top 2 most-likely tokens.

Model X transitions from c: $P(a|c)=0.1, P(b|c)=0.6, P(c|c)=0.2, P(|c)=0.1$

Step 1 — Extend <S> a c:

- Top 2: b (0.6), c (0.2)
- Beam 1: <S> a c b (prob 0.6)
- Beam 2: <S> a c c (prob 0.2)

Step 2 — Extend each:

From <S> a c b (transitions: $P(a|b)=0.3, P(b|b)=0.4, P(c|b)=0.2, P(|b)=0.1$):

- $\rightarrow$ <S> a c b b = $0.6 \cdot 0.4 = \mathbf{0.24}$
- $\rightarrow$ <S> a c b a = $0.6 \cdot 0.3 = \mathbf{0.18}$

From <S> a c c:

- $\rightarrow$ <S> a c c b = $0.2 \cdot 0.6 = 0.12$
- $\rightarrow$ <S> a c c c = $0.2 \cdot 0.2 = 0.04$

Keep top 2 across all candidates: <S> a c b b (0.24), <S> a c b a (0.18)

Worked Example: Greedy generation (2024 Q2 style)

Text: BCBBCBBABCCCCBBCCBCCB (20 chars)

Unigram counts: A: 1, B: 11, C: 8

Unigram greedy picks B every time → next 3 tokens: **B B B**

Bigram from last char B: $P(C|B)=0.5$, $P(B|B)=0.4$, $P(A|B)=0.1$

Greedy → C. From C: $P(B|C)=0.71$. Greedy → B. From B: $P(C|B)=0.5$. → C.

Next 3 tokens (greedy bigram): C B C

Worked Example: Bigram CAN'T generate (2024 Q2d)

A sequence the bigram can't generate: AA — $P(A|A) = 0$ (never seen).

Fix: add-k smoothing → every transition gets non-zero probability. Use sampling instead of greedy so the low-probability transition can occasionally be picked.

10. Text Classification

10.1 Problem Types

Type	Definition	Example
Binary	2 mutually exclusive	spam / not spam
Multi-class	>2 mutually exclusive	news topic
Multi-label	document can have multiple labels	movie genres (action AND comedy)

Common exam error: confusing multi-class with multi-label. If a recipe could be both “Italian” AND “pasta”, it’s **multi-label**.

10.2 Confusion Matrix & Metrics

	Predicted +ve	Predicted –ve
Actual +ve	TP	FN
Actual –ve	FP	TN

Metric	Formula	When it matters
Accuracy	$(TP + TN) / (TP + FP + TN + FN)$	balanced classes
Precision	$TP / (TP + FP)$	when FP is costly (spam filter)
Recall	$TP / (TP + FN)$	when FN is costly (cancer screening)
F1	$2PR / (P + R)$	balance P and R
Fβ	$(1 + \beta^2) \cdot PR / (\beta^2 P + R)$	weight recall ($\beta > 1$) or precision ($\beta < 1$)

Key intuitions:

- High accuracy is misleading with class imbalance
- F1 is harmonic mean of P and R — penalises imbalance
- **F1 always lies between min(P,R) and max(P,R)** — handy sanity check

10.3 Multi-Class Metrics

Method	How	Use when
Macro	unweighted mean across classes	all classes matter equally
Weighted	mean weighted by class support	reflects overall with imbalance
Micro	pool TP/FP/TN/FN across classes	overall granular performance

10.4 ROC & AUROC

Plot TPR (recall) vs FPR. Start bottom-left:

- Actual +ve → UP
- Actual -ve → RIGHT

AUROC: 1.0 perfect, 0.5 random, <0.5 worse than random.

10.5 Train/Val/Test & Cross-Validation

- **Training** — learn parameters
- **Validation/Dev** — tune hyperparameters
- **Test** — final evaluation, touched ONCE

K-fold CV: split into k folds; train on k-1, validate on 1, rotate. Use for small datasets.

10.6 Classifier Problems

Problem	Symptom	Fix
Noisy labels	low train AND test	clean data, robust loss
Underfitting	low train AND test	more capacity
Overfitting	high train, low test	regularisation, more data, early stopping
Class imbalance	model predicts majority	resampling, class weights, F1 instead of accuracy

Exam-style Q&A: Interpret train vs test metrics (2022 Q3d / 2025 Q2c)

1. **Overfitting** — System B's train (0.67/0.48) » test (0.51/0.42). Memorised training peculiarities.
2. **Generalisation** — System A: train (0.62/0.43) ≈ test (0.58/0.51). Modest improvement, holds out-of-sample.
3. **Significance** — only test-set significance matters for deployment.
4. **Recommendation:** prefer System A.

Exam-style Q&A: 4 effects of triplicated data (2025 Q2b)

1. **Class imbalance** — triplicated class becomes 3× more common
2. **Biased training** — model over-focuses on that class
3. **Test set leakage** — duplicates across splits → inflated metrics
4. **Unrealistic class distribution** — metrics don't reflect production

Exam-style Q&A: Anomalies in metrics table (2025 Q2c)

Common anomalies to scan for:

1. Wrong metric for the task (perplexity for classification)
2. Class count mismatch between splits
3. Confusion matrix numbers don't sum to stated total

4. Sample counts inconsistent with confusion matrix
5. Class distribution flipped between splits
6. Validation set larger than training set
7. F1 outside $[\min(P,R), \max(P,R)]$ range
8. Macro precision on binary problems
9. Train performance worse than validation (suspicious)
10. Perfect validation scores (data leakage)

11. Contextual Word Embeddings & Transformers

11.1 Three Types of Word Vectors

Type	Example	Length	Synonymy?	Polysemy?
Static sparse	IBM model	vocab size	yes	no
Static dense	word2vec, GloVe, SVD	~100-1000	yes	no
Contextual dense	ELMo, BERT, GPT	~768	yes	yes

Synonymy: “car” $\approx$ “automobile” (different words, same meaning) **Polysemy:** “bank” = river OR money (same word, different meanings)

Exam-style Q&A: Word2vec on polysemous words (2022 Q1f)

A: Nearest neighbours of “jaguar” would mix senses — “lion” AND “Ford” AND “Apache” (animal, car, helicopter). The vector is a meaningless average.

Downstream problem: For classification, ambiguous high-magnitude vectors dominate bag-of-vectors representations. Solution: use contextual embeddings (BERT) producing different vectors for different uses.

Exam-style Q&A: Function-word vectors unreliable (2022 Q1d)

A: Function words (“the”, “of”) co-occur with virtually every content word. Their context vectors are flat distributions over the entire vocabulary — every dimension similar. They have low information content, their cosine similarity to most words is uniformly moderate, and they drown out signal.

11.2 Self-Attention

Inputs: Query, Key, Value matrices from input vectors:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Intuition: Q asks “what am I looking for?”, K answers “what do I contain?”, V provides content. $Q \cdot K^T$ computes similarity $\rightarrow$ softmax $\rightarrow$ weights $\rightarrow$ weighted sum of V.

Multi-head attention: parallel attention with different W matrices — captures different relationships.

11.3 BPE (Byte Pair Encoding) — CRITICAL TOPIC

Training Algorithm

1. **Pre-tokenise** corpus into words — removes whitespace
2. **Initial vocab** = all unique characters
3. **Repeat until target vocab size:**
 - Find most frequent adjacent symbol pair
 - Merge → add as new symbol to vocab

Inference (applying learned rules)

1. Pre-tokenise input
2. Split each word into characters
3. Apply merge rules **in order, greedily, repeatedly**
4. Output subwords

Worked Example: Training BPE for “mississippi is pie” (2023 Q1c)

Target vocab size: 8

Words: [mississippi, is, pie]. Initial vocab = {m, i, s, p, e} (5).

Iter 1: count pairs. Most common: (i,s) = 3. New vocab = {m, i, s, p, e, is} (6). Corpus: m-is-s-is-s-i-p-p-i, is, p-i-e

Iter 2: Most common: (p,i) = 2. New vocab = {m, i, s, p, e, is, pi} (7). Corpus: m-is-s-is-s-i-p-pi, is, pi-e

Iter 3: Most common: (is,s) = 2. New vocab = {m, i, s, p, e, is, pi, iss} (8) ✓ STOP

Learned merge rules (in order): (i,s)→is, (p,i)→pi, (is,s)→iss

Worked Example: Applying BPE rules (2024 Q1c)

Rules: (i,s)→is, (p,i)→pi, (i,e)→ie, (p,ie)→pie, (p,pi)→ppi, (si,ppi)→sippi

Tokenise: mississippi pie

Word 1: mississippi → [m, i, s, s, i, s, s, i, p, p, i]

Apply (i,s)→is greedily:

- Position 1-2: (i,s) → [m, is, s, i, s, s, i, p, p, i]
- Position 3-4: (i,s) → [m, is, s, is, s, i, p, p, i]
- Position 5-6: (s,i) — no rule
- Position 6-7: (i,p) — no rule

Apply (p,i)→pi:

- Position 8-9: (p,i) → [m, is, s, is, s, i, p, pi]

Apply (p,pi)→ppi:

- Position 6-7: (p, pi) → [m, is, s, is, s, i, ppi]

Apply (si,ppi)→sippi: requires “si” token, doesn’t exist. Skip.

Result: [m, is, s, is, s, i, ppi]

Word 2: pie → [p, i, e]

- (p,i)→pi: [pi, e]
- No further rules fire.

Result: [pi, e]

Combined: [m, is, s, is, s, i, ppi, pi, e]

Exam-style Q&A: BPE vs rule-based, advantages each way (2023 Q1d)

BPE advantages over rule-based:

1. Handles out-of-vocabulary words by decomposing into known subwords
2. Language-agnostic — learns from data
3. Compact, fixed vocabulary
4. Captures morphological structure naturally

Rule-based advantages over BPE:

1. Interpretable and predictable
2. Preserves linguistic units (never breaks mid-morpheme)
3. No training data needed
4. Stable across deployments

Exam-style Q&A: Effect of wrong BPE vocab size (2024 Q1d)

Too low (e.g. 100):

- Most words split into many subwords
- Sequences become very long → memory/compute cost
- Loses semantic units

Too high (e.g. 100,000,000):

- Massive embedding matrix → memory blowup
- Most entries appear rarely → poor embeddings
- Approaches whole-word tokenisation, losing OOV handling
- Most rare words still get [UNK]

Exam-style Q&A: Is [UNK] needed for BPE? (2024 Q1b)

A: In principle, *almost* never — BPE’s base vocab includes individual characters, so any input string can be decomposed character-by-character.

However: if input contains characters outside training alphabet (emojis, unseen scripts), BPE will hit [UNK]. Unicode has 150,000+ characters; BPE training rarely covers all of them.

Byte-level BPE (BBPE) solves this completely — operates on raw UTF-8 bytes (256 possible values), the base vocab covers everything.

Conclusion: standard BPE rarely needs [UNK]; BBPE never does.

11.4 Special Tokens

Token	Purpose	Used by
[CLS]	classification — final hidden state used for whole-seq prediction	BERT
[SEP]	sentence separator	BERT
[PAD]	padding to batch length	most
[MASK]	masked token for MLM training	BERT
<S>, <E>	start/end of sequence	various LMs

11.5 Stopword Removal — BoW vs Transformer

Exam-style Q&A: Why remove stopwords in BoW but not in transformer? (2024 Q1a)

A: In **bag-of-words**, word order is lost — only counts matter. Stopwords (“the”, “of”, “is”) appear in nearly every document with high frequency, dominate the feature space, and contribute no discriminative signal. Removing them reduces noise and shrinks the feature space.

BoW example: Two news articles, sports and politics, both contain “the” hundreds of times. Including “the” wastes a dimension with zero classification signal.

In **transformers**, the model receives the full sequence with positional info and uses self-attention to weight each token dynamically. Stopwords carry crucial syntactic info: “I am the king” vs “I am king” mean different things. The model learns to attend or ignore as needed.

Transformer example: In “the cat sat on the mat”, the second “the” provides positional context. Removing would corrupt the input.

11.6 Transformer Encoder vs Decoder

Architecture	Example	Self-attention	Use case
Encoder-only	BERT	bidirectional	classification, NER, embeddings
Decoder-only	GPT	causal (only earlier tokens)	text generation
Encoder-decoder	T5, BART	encoder bidir, decoder causal + cross-attn	translation, summarisation

11.7 Pretraining + Fine-tuning

Pretraining objectives:

- **MLM** (BERT): mask 15% of tokens, predict them

- **Causal LM** (GPT): predict next token
- **Next Sentence Prediction** (BERT): does B follow A?
- **Spot corrupted word** (ELECTRA)

Why it works: language modelling has nearly unlimited data; learns general patterns transferable via fine-tuning on small labelled datasets.

Fine-tuning: keep most parameters, replace final layer(s) for downstream task.

12. POS Tagging & Parsing

12.1 Hidden Markov Models

Components:

- **Transitions:** $P(y_i|y_{i-1})$
- **Emissions:** $P(x_i|y_i)$

Greedy decoding

At each position, pick tag maximising $P(y_i|y_{i-1}) \cdot P(x_i|y_i)$. **Problem:** locally optimal $\neq$ globally optimal.

Viterbi algorithm (DP-based, optimal)

For each tag y at position i :

$$V[i][y] = \max_{y'} \text{ of } (V[i-1][y'] \cdot P(y|y') \cdot P(x_i|y))$$

$$\text{backptr}[i][y] = \text{argmax } y'$$

Backtrack from final position.

Complexity: $O(n \cdot |T|^2)$.

Exam-style Q&A: Why HMMs for POS tagging? (2023 Q3d)

A: HMMs model the *sequential dependency* between tags — adjectives precede nouns, etc. POS tags strongly depend on neighbours, so independent classifiers miss this signal. HMMs also handle *emissions* $P(\text{word}|\text{tag})$ capturing lexical preferences (“run” more likely verb than noun).

Viterbi finds globally optimal tag sequence in polynomial time — critical because greedy can propagate early errors. However, HMMs make strong independence assumptions, can’t use rich features (capitalisation, suffixes), and struggle with domain text unless retrained.

Exam-style Q&A: HMM fails on domain text (2022 Q3c)

A: The HMM was trained on general-domain text. On domain-specific text (patents, biomedical), it encounters unseen words — emissions default to [UNK] with no information. Domain text also uses standard words unusually (“claim” is noun in patents, verb in news) and has different transitions (heavy nominal compounds).

Fixes:

1. Fine-tune on small labelled in-domain corpus
2. Use transformer-based tagger pre-trained on the domain (SciBERT, BioBERT)
3. Add hand-crafted features (custom suffixes, term lists)
4. Self-training using high-confidence predictions

12.2 Constituency Parsing

Hierarchical phrases: NP, VP, PP. Trees show phrase nesting.

12.3 Dependency Parsing

Each word has a single head; directed graph, weakly connected, acyclic.

Transition-based parsing

Stack + buffer. Three actions:

- **Shift** — move next word from buffer to stack
- **Left-Arc** — edge from stack[top] to stack[top-1], pop stack[top-1]
- **Right-Arc** — edge from stack[top-1] to stack[top], pop stack[top]

Classifier picks action at each step.

13. Ethics in NLP

13.1 Key Issues

Issue	Description
Bias	Inherits training data biases (gender, race, geography)
Environmental cost	Massive energy consumption (GPT-3 $\approx$ 1287 MWh)
Transparency	Black-box models; hard for high-stakes use
Copyright	Training on copyrighted material; output reproduction
Liability	Who's responsible for harmful recommendations?
Education impact	Students using LMs for coursework
Privacy	Memorise and leak training data
Dual use	Disinformation, deepfakes, phishing

13.2 Best Practices

- **Datasheets for datasets** (Gebru et al.) — document collection, demographics, limitations
- **Model cards** (Mitchell et al.) — intended use, evaluation, known biases
- **Bias audits** — StereoSet, CrowS-Pairs
- **Carbon reporting**
- **Opt-out mechanisms**

Exam-style Q&A: Challenges of LM encoding knowledge (2023 Q3f)

1. **Stale knowledge** — only knows training-time facts; updating requires re-training or knowledge editing
2. **No source attribution** — produces fluent text but can't cite; can't distinguish memorised fact from confabulation

Other valid: privacy leakage, legal liability, biased knowledge

14. Information Extraction

14.1 Named Entity Recognition (NER)

IOB Tagging

- **I**nside an entity
- **O**utside
- **B**eginning (when two entities adjacent)

Example:

Tim	Cook	announced	Apple	in	Cupertino
B-PER	I-PER	O	B-ORG	O	B-LOC

Methods

Method	Description
CRF	Discriminative sequence model
BiLSTM-CRF	Neural + CRF (strong baseline)
Transformer (BERT)	Pretrained encoder + token classification (SOTA)

Exam-style Q&A: Nested entity mentions (2023 Q3e)

A: One entity span contained in another. Example: in “**University of Glasgow** Computing Science Department”, “**Glasgow**” is LOCATION nested in “University of Glasgow” which is ORGANISATION.

Standard IOB assumes flat spans. To handle nesting:

- Multiple tag layers (one per nesting level)
- Span-based models predicting (start, end, label) triples
- Graph-based models allowing overlapping spans

14.2 Entity Linking

Pipeline:

1. **NER** — find mentions
2. **Candidate generation** — dictionary lookup OR dense vector retrieval
3. **Reranking** (optional, high-accuracy) — transformer scores (mention, candidate) pairs

Example: “Apple” in “Apple released the iPhone” → Apple Inc. (not the fruit).

14.3 Relation Extraction

Method	Description	Pros	Cons
Co-occurrence	Count entities appearing together	simple	no relation type
Rule-based	Patterns (“X, who starred in Y”)	high precision	low recall, brittle

Method	Description	Pros	Cons
BERT-based	Insert entity markers, classify	high accuracy	needs labelled data

Entity markers: [E1]Tim Cook[/E1] is CEO of [E2]Apple[/E2].

14.4 Coreference Resolution

Which mentions refer to the same entity?

- **Anaphora** — backward (“Mary went... She bought milk”)
- **Cataphora** — forward (“After she sat, Mary ordered”)
- **Exophora** — outside text (“Look at this!”)

Methods:

1. Mention detection (pronouns, NEs, NPs)
2. Coreference clustering
3. Hobbs algorithm — classical syntactic approach

15. Large Language Models

15.1 Training Pipeline

1. Pretraining → next-token prediction on huge corpus
2. Instruction tuning → supervised on (instruction, response) pairs
3. RLHF → reinforcement learning from human feedback
4. Safety fine-tuning → reduce harmful outputs

RLHF:

1. Collect human preference pairs
2. Train reward model to predict preferences
3. PPO fine-tune LM to maximise reward

15.2 Prompt Engineering

Technique	Description
Zero-shot	Just describe the task
Few-shot	Include examples in prompt
Chain-of-Thought	“Let’s think step by step”
Self-consistency	Sample multiple CoT, take majority
Explicit instructions	“Answer in JSON. Use only these keys...”

15.3 LLM Issues

Issue	Mitigation
Hallucination	RAG, fact-checking, lower temperature
Data contamination	newer benchmarks, manual examples
Cost	smaller fine-tuned models, distillation
Latency	speculative decoding
Bias	RLHF, debiasing prompts, audits
Privacy leakage	differential privacy in training

Exam-style Q&A: Rank LogReg, BERT, LLaMA by cost (2025 Q3a)

A: Logistic Regression < BERT-base < LLaMA-70B.

LogReg = linear over sparse features, millions of params, CPU in ms. BERT-base = ~110M params, single GPU. LLaMA-70B = ~70 billion (≈600× more), requires multiple high-end GPUs even for inference.

Exam-style Q&A: Zero-shot vs data contamination (2024 Q3a)

A: Before claiming “zero-shot,” verify the LLM wasn’t trained on test data. LLMs pretrain on massive web crawls that often include public benchmark datasets — **data contamination**. If test set appeared in pretraining, the model has effectively *seen* the data; result isn’t truly zero-shot. Check training data documentation; consider whether dataset is well-known enough to have been crawled.

Exam-style Q&A: BERT predicts “positive” on brand name (2025 Q3d)

A: Brand “BeStep” not in BERT’s vocabulary as a single token → sub-word tokeniser splits it into best + ##ep. Model sees “best” in every brand mention and learns to associate with positive sentiment.

Fix: replace brand with placeholder before classification: `text = text.replace("BeStep", "[BRAND]")`. Or add “BeStep” as new vocab token and fine-tune.

Exam-style Q&A: Cross-lingual classification limits (2025 Q3c)

LogReg problem: Classifier learned weights for English tokens only. Other-language tokens map to [UNK] or zero vectors → drastically reduced performance.

BERT-base problem: Sub-tokeniser is English-biased. Other-language words split into many sub-tokens (per-character), producing degenerate representations.

Solution that addresses both: Use a **multilingual pretrained model** (mBERT, XLM-R) trained on 100+ languages. Its tokeniser knows sub-tokens from many scripts; pretraining aligns embeddings across languages.

Exam-style Q&A: Counter-arguments to LLM-only labelling (2024 Q3b)

1. **$\kappa = 0.61$ IS reliable** by standard interpretation (Landis & Koch: 0.61–0.80 = substantial). Premise is wrong.
2. **LLMs have their own biases** — trained on biased text; lack the diversity of multiple human annotators.
3. **Circular evaluation** — using an LLM for labels AND an LLM for the classifier essentially asks “does LLM agree with itself?” Won’t generalise to deployment.

Additional: data contamination risk; cost at scale; reproducibility (LLM versions change).

16. Latest Research (Lecture 10)

16.1 Retrieval-Augmented Generation (RAG)

Problem: LLMs hallucinate; knowledge frozen at training.

Solution: retrieve relevant documents from external corpus at inference time, concatenate with question, generate grounded answer.

Pipeline: Query → Retriever → top-k passages → LLM → grounded answer

Benefits:

- Updatable knowledge
- Source attribution
- Smaller LMs match larger ones with good retrieval
- Less hallucination

Architectures:

- **Fusion-in-Decoder (FiD)** — encode passages separately, concat before decoder
- **RETRO (DeepMind)** — cross-attention to retrieved chunks
- **REALM** — joint retriever-reader training

Limitations: still hallucinates from suggestive prompts; retrieval quality bottleneck; lost-in-the-middle.

16.2 Lost in the Middle

When given many passages, LLMs show U-shaped accuracy: high at start/end of context, low in middle. Don't rely on long-context LLMs to extract middle-passage facts without explicit prompting.

16.3 Tool Use

LLMs invoke external tools (calculators, search APIs, translators, code).

Mechanism: LM generates special tokens like [Calculator(2+2) → 4]. Generation pauses, tool executes, result inserted, generation resumes.

Toolformer example: "Out of 1400 participants, 400 (or [Calculator(400/1400) → 0.29] 29%) passed."

16.4 Knowledge Editing

Full retraining too expensive. Approaches:

- **Minimal weight edits** — ROME, MEMIT
- **External memory** — store edits in key-value store
- **Adapter modules** — small trainable modules on frozen base

16.5 Reasoning

CoT prompting improves multi-step problems. Latest models (o1, R1) trained specifically for hidden reasoning chains. Massive token cost increase, but improves hard reasoning.

16.6 Agentic AI

Chain multiple LLM calls:

1. **Planner** — break goal into subtasks
2. **Worker** — execute (possibly using tools)
3. **Critic** — evaluate, decide next action
4. **Loop** until done or stuck

Strengths: complex multi-step tasks, iterative refinement. **Weaknesses:** can degrade into nonsense, hard to debug, expensive.

16.7 Predicted Q3 synthesis question

Q (predicted): Building an LLM-powered assistant for a healthcare company. Discuss issues and mitigations.

Model answer outline:

1. **Hallucination** — RAG with curated medical literature; require citations; refuse on low retrieval confidence.
2. **Stale knowledge** — RAG ensures up-to-date answers; flag knowledge cutoff.
3. **Liability/transparency** — disclaimers, source links, human-physician review for high-risk.
4. **Privacy** — on-prem deployment or zero-retention API; PII redaction.
5. **Bias** — evaluate per-demographic; diverse expert reviewers.
6. **Test contamination** — fresh hold-out sets with domain experts.
7. **Lost in the middle** — retrieval to focus on relevant record sections.

17. Exam Strategy

17.1 Time Budget (90 minutes)

Activity	Time
Scan all 3 questions	3 min
Q1	~27 min
Q2	~27 min
Q3	~27 min
Review	6 min

Roughly **1.3 min per mark**.

17.2 Mark-Detail Calibration

“Each mark is roughly one idea or one point in the answer.” — Sean MacAvaney

Marks	Expected length
1	1 sentence
2	2 sentences OR 1 paragraph
3–4	structured paragraph with example
5–8	multiple paragraphs OR bullet list with explanation

Common failures:

- Too little detail on a 4-mark question
- Too much detail on a 2-mark question (wastes time, can include errors)

17.3 Answer Templates

“Discuss N problems with...” → numbered bullets, each = 1 problem + 1-sentence justification.

“Justify whether you would use X for Y...” → (1) decision, (2) reasoning principle, (3) domain-specific example.

“Calculate...” → (1) formula, (2) substitute values explicitly, (3) intermediate arithmetic, (4) final answer.

“Critique this response...” → (1) identify errors/omissions, (2) provide correct version, (3) affirm correct parts briefly.

17.4 Pitfalls to Avoid

1. Copy-pasting from crib sheet (explicitly mentioned as not scoring)
2. Not showing work on calculations
3. Vague generic answers without domain examples
4. Going over time on early questions

5. Answering more sub-parts than asked
6. Confusing similar concepts (multi-class vs multi-label, encoder vs decoder, macro vs micro)

17.5 Final-Week Revision Plan

- **Day 1-2:** Re-do all past paper Q1 sub-parts. Focus on BPE, set similarity, metric properties.
- **Day 3-4:** Re-do all past paper Q2 sub-parts. Practise beam search until automatic; 2 perplexity calculations/day.
- **Day 5-6:** Read Lectures 7, 8, 9, 10. Write your own 3-sentence summary of each topic. Practise BERT/LLM scenario questions.
- **Day 7:** Cold simulate the exam — pick one past paper, attempt under 90-min timed conditions, mark against sample answers.

Appendix A — Critical Formulas at a Glance

Similarity:

- Overlap = $|X \cap Y| / \min(|X|, |Y|)$
- Dice = $2|X \cap Y| / (|X| + |Y|)$
- Jaccard = $|X \cap Y| / |X \cup Y|$
- Cosine = $(\vec{x} \cdot \vec{y}) / (\|\vec{x}\| \cdot \|\vec{y}\|)$
- Euclidean = $\sqrt{\sum (x_i - y_i)^2}$

TF-IDF:

- TF (log) = $1 + \log_{10}(tf)$ if $tf > 0$
- IDF = $\log(N/df)$
- TF-IDF = $(1 + \log(tf)) \cdot \log(N/df)$

Language Model:

- Add-k = $(count + k) / (\sum count + k|V|)$
- Perplexity = $(\prod P(w_i))^{-1/n} = 2^{\text{Cross-Entropy}}$
- Cross-entropy = $-(1/n) \sum \log_2 P(w_i)$

Classification:

- Precision = $TP / (TP + FP)$
- Recall = $TP / (TP + FN)$
- F1 = $2PR / (P + R)$
- Accuracy = $(TP + TN) / \text{total}$

Attention:

- Attention = $\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$

Appendix B — Topic to Lecture to Likely Question

Lecture	Title	Key concepts	Likely exam slot
1	Intro, Tokens, Doc Similarity	NLP pipeline, encodings, set sim, n-grams	Q1
2	Geometric Sim, Distributions, Clustering	TF-IDF, cosine, K-Means	Q1
3	Language Modelling	n-grams, smoothing, perplexity, beam search	Q2
4	Text Classification	metrics, splits, overfitting	Q2
5	Contextual Word Embeddings	BPE, BERT, attention	Q1 (BPE) or Q3 (BERT)

Lecture	Title	Key concepts	Likely exam slot
6	POS Tagging & Parsing	HMM, Viterbi	Q3
7	Ethics	bias, transparency, environmental cost	Q3 sub-part
8	Information Extraction	NER, entity linking, relation extraction	Q3
9	Large Language Models	RLHF, prompt engineering, hallucination	Q3
10	Latest Research	RAG, agentic, tool use	Q3 sub-part

Appendix C — Highest-ROI Revision Priorities (Ranked)

1. **BPE algorithm** — can you trace through rule application by hand? (Q1)
2. **Set similarity formulas + metric properties** — semi-metric, triangle inequality counter-examples (Q1)
3. **Beam search with beam width 2** — practise on bigram models until automatic (Q2)
4. **Smoothing (add-k, interpolation)** — why, when, downsides (Q2)
5. **Perplexity calculation** — formula + step-by-step (Q2)
6. **BERT classifier failure modes** — sub-word tokenisation, domain shift, cross-lingual (Q3)
7. **LLM issues** — hallucination, RAG, prompt engineering, data contamination (Q3)
8. **HMM Viterbi for POS tagging** — overdue, worth reviewing (Q3 backup)
9. **Information Extraction pipeline** — NER + Entity Linking + Relation Extraction (Q3 backup)
10. **K-Means** — only review if time permits (Q1 backup)

End of Complete Study Guide. Estimated total study time: 15–20 hours over 1 week, plus 4–6 hours of past-paper practice. Good luck on the exam.